

ETHENA · SECURITY REVIEW

Ethena

Execution Guard Review.

A security review of Execution Guard Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ethena	Execution Guard Review	Guardian	March 23 – March 25, 2026	April 2, 2026

§ 00

Contents.

24 pages — overview, confidence ranking, scope, 12 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 12 findings	P. 08
§ 05	Disclaimer	P. 23
§ 06	About Guardian	P. 24

§ 01

Overview.

Guardian reviewed Execution Guard Review across a single review round. The final report includes 2 Medium, 4 Low, 6 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
Ethena	
PROJECT	
Execution Guard Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
March 23 – March 25, 2026	
FINAL REPORT	
April 2, 2026	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	2	0	1	0	1
Low	4	0	2	0	2
Informational	6	0	2	0	4
All severities	12	0	5	0	7

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Execution Guard Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Guard Update Timelock Bypass	Validation	MEDIUM	RESOLVED	p. 8
M-02	delegatecall batching bypasses Safe guard checks	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
L-01	setFallbackHandler Can Bypass Executor Whitelist	Validation	LOW	RESOLVED	p. 12
L-02	No Module Check On Safe Onboarding	Validation	LOW	ACKNOWLEDGED	p. 13
L-03	Ownership Transfer Not Timelocked	Access Control	LOW	ACKNOWLEDGED	p. 14
L-04	Unrestricted renounceOwnership	Access Control	LOW	RESOLVED	p. 15
I-01	Redundant No-Op Parameter References	Superfluous Code	INFORMATIONAL	RESOLVED	p. 16
I-02	Timelock Reschedule Overwrites Existing Entry	Validation	INFORMATIONAL	RESOLVED	p. 18
I-03	Stale Timelocks Persist After Safe Removal	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-04	Missing Event For Initial Timelock Delay	Events	INFORMATIONAL	RESOLVED	p. 20
I-05	Delist Fails On Unreadable Guard Status	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-06	No Fallback Handler For Interface Changes	Compatibility	INFORMATIONAL	RESOLVED	p. 22

MEDIUM **M-01**

RESOLVED

Guard Update Timelock Bypass

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/main/src/EthenaSafeGuard.sol#L258-L317>, <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/main/src/EthenaSafeGuard.sol#L465-L472>

REVIEW Main Review

DESCRIPTION

`_isGuardUpdateOperation` uses a strict equality check (`data.length == 36`) to identify `setGuard` calls:

```
function _isGuardUpdateOperation(address safe, address to, bytes memory data, Enum.Operation
operation)
internal
pure
returns (bool)
{
    return to == safe && operation == Enum.Operation.Call && data.length == 36
    && _selector(data) == IGuardManager.setGuard.selector;
}
```

After identifying the `setGuard` call, it is checked against a timelock that must have been set prior to execution. However, by appending a trailing byte to the calldata (i.e., 37 bytes instead of 36), the `_isGuardUpdateOperation` check will return false. The transaction then falls through to the executor whitelist path in `checkTransaction` and executes without timelock validation. The EVM's ABI decoder ignores the extra byte, so `setGuard` executes normally on the Safe.

This allows a guard rotation (including replacement with a malicious guard) in a single transaction with no on-chain observation window, breaking the system's core timelocked config update invariant. Anyone relying on the timelock delay for monitoring are left with no time to respond.

RECOMMENDATION

Replace the strict length check with a minimum bound in `_isGuardUpdateOperation`:

```
return to == safe && operation == Enum.Operation.Call && data.length >= 36
&& _selector(data) == IGuardManager.setGuard.selector;
```

M-01 Guard Update Timelock Bypass

CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/execution-guard-team1-1773954142760/tree/POC_GUARD_TIMELOCK_BYPASS

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

ACKNOWLEDGED

delegatecall batching bypasses Safe guard checks

CATEGORY Logical Error
LOCATION EthenaSafeGuard.sol
REVIEW Main Review

DESCRIPTION

The guard only reasons about the outer Safe transaction

In Safe v1.5.0, `execTransaction` verifies signatures, calls `guard.checkTransaction()` once for the outer tx, then executes the requested CALL or DELEGATECALL,

It does not re-run the guard on any inner calls emitted by code running under that outer `delegatecall`

Separately, Safe authorized modifier only checks `msg.sender = address(this)`, and Safe own `setGuard / enableModule` are authorized selfcall functions.

That becomes fatal when a generic `delegatecall` target is on `delegateCallAllowlist`

most notably Safe standard batcher `MultiSendCallOnly`, which Safe SDK uses for batched `onlyCalls` transactions.

when `MultiSendCallOnly` is `delegatecalled`, it can emit arbitrary CALL, and it maps to `= address(0)` to `address(this)`, the Safe itself.

So an outer tx that looks harmless to the guard (`DelegateCall` to an allowlisted target) can hide inner Safe self calls that the guard never classifies

The cleanest exploit is post onboarding module enablement, the guard blocks direct `enableModule/disableModule`, but it only checks the outer calldata shape

A whitelisted executor with valid Safe signatures can submit one outer tx

1- `DelegateCall` into allowlisted `MultiSendCallOnly`

2- inner subcall A: self-call `enableModule(attackerModule)`

3- inner subcall B: call `attackerModule.attack()` Once enabled, a Safe module has effectively unlimited execution power,

and `execTransactionFromModule` only checks that the caller is an enabled module plus any separate module guard

M-02 delegatecall batching bypasses Safe guard checks

CONTINUED

RECOMMENDATION

Consider making delegatecall unsupported for guarded Safes

```
if (operation == Enum.Operation.DelegateCall) {  
  revert DelegateCallNotAllowed(to);  
}
```

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/c3cd9289dc44f0be73590f1af6fdf037>

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

setFallbackHandler Can Bypass Executor Whitelist

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eccc/src/EthenaSafeGuard.sol#L297-L300>

REVIEW Main Review

DESCRIPTION

The guard enforces that a whitelisted executor must approve all operations through `checkTransaction`. The only exceptions are `setGuard(address(0))` execution and scheduling the emergency disable. Everything else requires an explicitly whitelisted `msgSender`.

However, `setFallbackHandler` is not blocked by the guard. Once a fallback handler is set, the Safe's `fallback()` [forwards](<https://github.com/safe-fndn/safe-smart-account/blob/main/contracts/base/FallbackManager.sol#L46-L87>) unrecognized calls to the handler via call with `msg.sender` as the Safe. This creates a path for operations to be executed as the Safe without flowing through `execTransaction` and therefore without any executor whitelist validation.

For example, if the handler is set to the guard's address, anyone can call guard functions like `scheduleTimelockedOperation` or `cancelTimelockedOperation` as the Safe, bypassing the requirement that a whitelisted executor must approve the operation. Setting the handler requires a guarded transaction through a whitelisted executor, but once configured, the bypass persists until the handler is reset.

RECOMMENDATION

Block `setFallbackHandler` in `checkTransaction` alongside module management (also ensure it is not pre-set):

```
bytes4 selector = _selector(data);
return selector == IModuleManager.enableModule.selector
|| selector == IModuleManager.disableModule.selector
|| selector == IFallbackManager.setFallbackHandler.selector;
```

RESOLUTION

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

No Module Check On Safe Onboarding

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eecc/src/EthenaSafeGuard.sol#L128-L143>

REVIEW Main Review

DESCRIPTION

`setSafeAllowed` does not verify that a Safe has zero modules enabled before adding it to the allowlist. The execution guard explicitly blocks `enableModule` and `disableModule` operations during `checkTransaction` to prevent modules from being added. However, a Safe with a pre-existing module can be onboarded without restriction.

A module on a Safe executes through `execTransactionFromModule`, which does not invoke the transaction guard's `checkTransaction` hook, allowing the safe to bypass every protection the guard provides: executor whitelist, timelock enforcement, delegatecall restrictions, and module management blocks. Additionally, if a module does exist, the guard's block on `disableModule` prevents remediation through normal guarded execution, forcing the Safe to go through the emergency guard removal path first.

The project documentation acknowledges that pre-enabled modules remain an independent execution path outside the guard's scope. However, enforcing this on-chain via a module check in `setSafeAllowed` would be consistent with the existing `_isSafeUsingThisGuard` pattern, rather than relying on off-chain diligence.

RECOMMENDATION

When adding a Safe to the allowlist, verify it has no modules enabled:

```
if (flags[i]) {  
  (address[] memory modules,) = ISafe(safes[i]).getModulesPaginated(address(0x1), 1);  
  if (modules.length > 0) revert SafeHasModulesEnabled(safes[i]);  
}
```

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Ownership Transfer Not Timelocked

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eecc/src/EthenaSafeGuard.sol#L15>

REVIEW Main Review

DESCRIPTION

Ownership transfer via `Ownable2Step` has no timelock, despite being the most privileged operation in the system. A new owner can immediately call `setTimelockDelay`, `setSafeListManager`, and schedule timelocked operations for `setWhitelist` and `setDelegateCallAllowlist`. The two-step transfer (`transferOwnership` + `acceptOwnership`) requires explicit acceptance but provides no enforced delay, both calls can land in consecutive blocks.

This is inconsistent with the system's design, since executor whitelist and delegatecall allowlist updates are timelocked despite already requiring `onlyOwner`, specifically to give external observers a reaction window. Ownership transfer is more powerful than any of those operations yet has no such window.

RECOMMENDATION

Apply the existing timelock mechanism to ownership transfers. `transferOwnership` could be routed through `scheduleTimelockedOperation` so that external monitors have the same visibility window they get for whitelist and delegatecall updates.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Unrestricted renounceOwnership

CATEGORY	Access Control
LOCATION	https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/main/src/EthenaSafeGuard.sol#L14
REVIEW	Main Review

DESCRIPTION

`EthenaSafeGuard` inherits `Ownable2Step` for safe two-step ownership transfers, but does not override `renounceOwnership()` from the base `Ownable` contract. If invoked, ownership is set to `address(0)`, permanently freezing all `onlyOwner` functions, including setting the executor whitelist, delegatecall allowlist, timelock delay, and safe list manager roles.

RECOMMENDATION

Override `renounceOwnership` to always revert.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Redundant No-Op Parameter References

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/main/src/EthenaSafeGuard.sol#L258-L284
REVIEW	Main Review

DESCRIPTION

`checkTransaction` names its unused parameters with a trailing underscore (i.e., `value_`, `signatures_`) and then references each as a no-op statement (`value_;`, `signatures_;`) to suppress unused variable compiler warnings. Both patterns become unnecessary when parameter names are simply omitted from the function signature, so using both is not needed and causes code clutter.

```
function checkTransaction(  
    address to,  
    uint256 value_, // named with underscore  
    bytes memory data,  
    ...  
) external override onlyAllowedSafe {  
    value_; // and also referenced as no-op  
    safeTxGas_;  
    ...  
}
```

RECOMMENDATION

Consider omitting parameter names entirely for unused arguments and remove the no-op references:

```
function checkTransaction(  
    address to,  
    uint256, // value  
    bytes memory data,  
    Enum.Operation operation,  
    uint256, // safeTxGas  
    uint256, // baseGas  
    uint256, // gasPrice  
    address, // gasToken
```

I-01 Redundant No-Op Parameter References

CONTINUED

```
address payable, // refundReceiver
bytes memory, // signatures
address msgSender
) external override onlyAllowedSafe {
```

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Timelock Reschedule Overwrites Existing Entry

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/main/src/EthenaSafeGuard.sol#L175>

REVIEW Main Review

DESCRIPTION

`scheduleTimelockedOperation` unconditionally overwrites `timelockUnlockAt` if called again with the same operation parameters. If an operation is already scheduled, even within its valid execution window, a new call resets the `unlockTimestamp` to `block.timestamp + timelockDelay`. This could cause confusion for off-chain observers if, for example, the owner reduces `timelockDelay` via `setTimelockDelay` and then reschedules a pending operation with a shorter window than what was originally set.

RECOMMENDATION

Within `scheduleTimelockedOperation`, revert if a timelock entry already exists for the given operation.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

ACKNOWLEDGED

Stale Timelocks Persist After Safe Removal

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eecc/src/EthenaSafeGuard.sol#L133-L142
REVIEW	Main Review

DESCRIPTION

When a Safe is removed from `allowedSafes` via `setSafeAllowed`, its pending timelock entries in `timelockUnlockAt` are not cleared. If the Safe is re-onboarded within the `TIMELOCK_EXPIRY` window (7 days), previously scheduled operations can be executed immediately without waiting through a new delay period.

RECOMMENDATION

Document that re-onboarding a recently delisted Safe may carry stale timelock entries, or consider cancelling any pending timelocked operations as part of the offboarding process.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

Missing Event For Initial Timelock Delay

CATEGORY Events

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eecc/src/EthenaSafeGuard.sol#L69>

REVIEW Main Review

DESCRIPTION

The constructor sets `timelockDelay = DEFAULT_TIMELOCK_DELAY` but does not emit a `TimelockDelayUpdated` event. Every other state change in the constructor emits its corresponding event (`SafeListManagerUpdated`, `ExecutorWhitelistUpdated`, `SafeAllowedUpdated`, `DelegateCallTargetUpdated`). Anyone relying on events to track the timelock delay would not capture the initial value.

RECOMMENDATION

Emit `TimelockDelayUpdated` in the constructor:

```
timelockDelay = DEFAULT_TIMELOCK_DELAY;  
emit TimelockDelayUpdated(0, DEFAULT_TIMELOCK_DELAY);
```

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

ACKNOWLEDGED

Delist Fails On Unreadable Guard Status

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eccc/src/EthenaSafeGuard.sol#L520-L531 , https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eccc/src/EthenaSafeGuard.sol#L137
REVIEW	Main Review

DESCRIPTION

`_isSafeUsingThisGuard` reverts unconditionally if the `getStorageAt` call fails or returns unexpected data. If a Safe's guard storage becomes unreadable for any reason, the manager cannot remove it from the allowlist, and the Safe becomes permanently stuck in `allowedSafes`.

```
function _isSafeUsingThisGuard(address safe) internal view returns (bool) {
    try IStorageAccessible(safe).getStorageAt(SAFE_GUARD_STORAGE_SLOT, 1) returns (bytes memory rawGuardWord) {
        if (rawGuardWord.length != 32) revert SafeGuardStatusUnavailable(safe);
        address configuredGuard = abi.decode(rawGuardWord, (address));
        return configuredGuard == address(this);
    } catch {
        revert SafeGuardStatusUnavailable(safe); // blocks delist
    }
}
```

The project documentation acknowledges the fail-closed behavior as a safety tradeoff, preferring temporary delist unavailability over unsafe delisting. However, if a Safe's guard storage becomes permanently unreadable, there is no override path. The Safe remains in `allowedSafes` indefinitely with no mechanism to remove it.

RECOMMENDATION

If guard status cannot be determined, allow removal rather than blocking it. Alternatively, consider providing an owner-only emergency delist that bypasses the guard status check as a last resort.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

RESOLVED

No Fallback Handler For Interface Changes

CATEGORY Compatibility

LOCATION <https://github.com/GuardianOrg/execution-guard-team1-1773954142760/blob/23c7b48e73d03d73e91a98475925df624923eecc/src/EthenaSafeGuard.sol#L15>

REVIEW Main Review

DESCRIPTION

`EthenaSafeGuard` does not implement a `fallback()` function. If the Safe is upgraded to a version that changes the guard interface (e.g., different `checkTransaction` signature), calls to the guard would revert. Since the emergency disable path (`setGuard(address(0))`) also flows through `checkTransaction`, the Safe could become permanently locked.

Other guard implementations (Gnosis `ScopeGuard`, Yearn `StealthSafeGuard`) include a non-reverting fallback specifically to prevent this scenario:

```
fallback() external {  
    // We don't revert on fallback to avoid issues in case of a Safe upgrade  
    // E.g. The expected check method might change and then the Safe would be locked.  
}
```

RECOMMENDATION

Consider adding a non-reverting `fallback()` to avoid permanently locking guarded Safes in the event of a Safe interface upgrade.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.